PolyFlop8

Data Path Integration Test Card

TS-ST-DP-001

Verification of ALU + Register File + Status Register Interaction

Test Plan ID:	TS-ST-DP-001
Revision:	1.0
Date:	January 6, 2026
Test Stage:	Stage 2 - Data Path Integration
Author:	Artem Horiunov
Status:	APPROVED
Coverage Target:	90% Path Coverage

Based on PolyFlop8 Test Architecture Overview v1.0 & ISTQB Requirements v1.0

Contents

1 TEST OVERVIEW

1.1 Purpose

This document defines the test cases for Stage 2: Data Path Integration testing of the PolyFlop8 microcontroller. This stage verifies the interaction between the ALU, Register File, and Status Register (SREG), forming the execution core of the processor.

1.2 Scope

- Verification of ALU → Register File data path
- Verification of Register File → ALU operand supply
- Verification of ALU → SREG flag propagation
- Verification of SREG → Register File restore mechanism
- Arithmetic pipeline validation
- Flag generation and persistence checking
- **Exclusions:** Instruction fetch, memory subsystem, I/O operations

1.3 Test Configuration

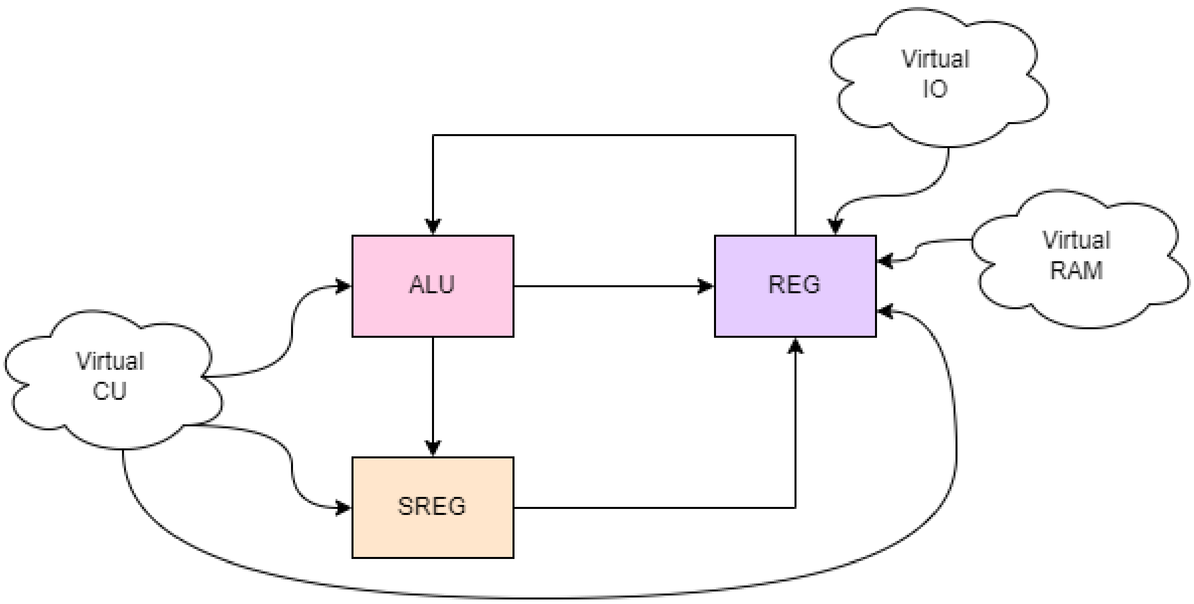

Figure 1: Data Path Integration Test Configuration (from TS-ST-Overview-001)

Test Environment:

- Virtual sequencer acts as surrogate Control Unit
- Direct stimulus injection to ALU control signals
- Bypassed instruction fetch logic
- Manual register file access through BFM

1.4 Key Interfaces Under Test

Interface	Test Focus	Covered Requirements
ALU → RegFile	Result write-back mechanism	TR-ALU-01, TR-REG-02
RegFile → ALU	Operand selection and bypass	TR-REG-01, TR-ALU-03/04
ALU → SREG	Flag generation and latching	TR-ALU-02, TR-SREG-01
SREG → RegFile	Status restore (POP SREG)	TR-SREG-02
Control → ALU/RegFile	Control signal timing	TR-CU-03, TR-CU-05

2 TEST CASES

2.1 Test Case DP-01: Register Read/Write Integrity

Test ID	DP-01
Priority	P1
Title	Register Read/Write Integrity (RegFile isolation)
Objective	Verify asynchronous read and synchronous write operations to all 8 registers
Requirements	TR-REG-01, TR-REG-02
Pre-conditions	Reset applied. All registers initialized to 0x00. Virtual Control Unit configured to bypass normal pipeline.
Test Steps	1. Set register file write enable (we='1') 2. Set write address (addr_w=R1) and data input (data_in=0x55) 3. Apply clock pulse (rising edge) 4. Disable write enable (we='0') 5. Set read address (addr_a=R1) 6. Monitor data_a output 7. Repeat for all registers (R0-R7) with different values
Expected Results	• After step 3: Register R1 contains 0x55 • After step 6: data_a outputs 0x55 • Other registers (R0, R2-R7) remain at 0x00 • No unintended writes during we='0'
Pass Criteria	All register writes and reads execute correctly without corruption
Coverage Points	Register write timing, address decoding, data retention

2.2 Test Case DP-02: ALU-Register Loopback

Test ID	DP-02
Priority	P1
Title	ALU-Register Loopback (Result propagation)
Objective	Verify complete data path: Register → ALU → Register
Requirements	TR-ALU-01, TR-ISA-01, TR-REG-02
Pre-conditions	R1 = 0x0A (10), R2 = 0x14 (20), ALU configured for ADD operation
Test Steps	1. Set ALU operation: alu_sel="0000" (ADD) 2. Configure operand muxes: alu_mux_sel_a='0', alu_mux_sel_b='0' 3. Set register addresses: addr_a=R1, addr_b=R2 4. Enable register write: we='1', addr_w=R3 5. Apply clock pulse 6. Monitor alu_result bus and R3 contents
Expected Results	• During step 4: alu_result = 0x1E (30) • After step 5: Register R3 = 0x1E • Registers R1, R2 unchanged • SREG flags: Z=0, N=0, C=0, V=0, H=0
Pass Criteria	Correct arithmetic result propagates through entire data path
Coverage Points	ALU operation, operand selection, result write-back

2.3 Test Case DP-03: Flag Persistence

Test ID	DP-03
Priority	P2
Title	Flag Persistence (SREG retention checks)
Objective	Verify SREG flags persist through non-flag-modifying operations
Requirements	TR-SREG-04, TR-ALU-02
Pre-conditions	R1 = 0x0A, R2 = 0x14, SREG initialized to 0x00
Test Steps	1. Execute SUB R1, R2 (10 - 20 = -10) 2. Observe SREG flags 3. Execute NOP (MOV R0, R0) 4. Check SREG flags again 5. Execute ADD R1, R2 with result that doesn't change flags 6. Verify flag persistence
Expected Results	• After step 1: N=1 (negative), C=1 (borrow), Z=0, V=0 • After step 3: Flags unchanged (N=1, C=1, Z=0, V=0) • After step 5: Flags remain unchanged • Bits I and T (bits 6-7) remain at initial values
Pass Criteria	Flags persist correctly through non-modifying operations
Coverage Points	SREG latching logic, flag preservation

2.4 Test Case DP-04: Carry Propagation

Test ID	DP-04
Priority	P2
Title	Carry Propagation (ADC/SBC chain verification)
Objective	Verify carry flag correctly propagates through multi-byte operations
Requirements	TR-ISA-01 (ADC), TR-ALU-02
Pre-conditions	C flag = '1', R1 = 0x05, R2 = 0x05
Test Steps	1. Set ALU operation: alu_sel="0001" (ADC) 2. Configure carry input: cin='1' 3. Execute ADC R1, R2 4. Monitor result and C flag 5. Test SBC with borrow propagation 6. Test multi-byte addition sequence
Expected Results	• After step 3: Result = 0x0B (5 + 5 + 1 = 11) • C flag = 0 (no carry out) • Z=0, N=0, V=0 • SBC correctly subtracts with borrow
Pass Criteria	Carry/borrow correctly included in arithmetic operations
Coverage Points	Carry chain logic, multi-byte operation support

2.5 Test Case DP-05: Immediate Data Path

Test ID	DP-05
Priority	P2
Title	Immediate Data Path
Objective	Verify immediate value routing through ALU bypass
Requirements	TR-ISA-08, TR-ALU-04
Pre-conditions	RS_IMM input connected to test sequencer
Test Steps	1. Set RS_IMM input = 0xFF 2. Configure ALU: alu_mux_sel_b='1' (select immediate) 3. Set ALU operation: alu_sel="1000" (LDI - pass A) 4. Set alu_mux_sel_a='0' (select any register) 5. Enable register write to R1 6. Apply clock pulse 7. Verify result
Expected Results	• After step 6: Register R1 = 0xFF • ALU correctly passes immediate value • No flags affected (LDI doesn't set flags)
Pass Criteria	Immediate values correctly routed to destination
Coverage Points	Immediate bus routing, ALU pass-through operations

2.6 Test Case DP-06: X-Pointer Interface

Test ID	DP-06
Priority	P3
Title	X-Pointer (Indirect Addressing)
Objective	Verify R7 hardwired output for X-pointer indirect addressing
Requirements	TR-REG-03, TR-ISA-12
Pre-conditions	Register file initialized
Test Steps	1. Load value 0xAA into R7 2. Monitor data_r7x output port 3. Change R7 to 0x55 4. Monitor data_r7x again 5. Verify other registers don't affect data_r7x
Expected Results	• After step 1: data_r7x = 0xAA • After step 3: data_r7x = 0x55 • Only R7 contents appear on data_r7x • Changes to R0-R6 don't affect data_r7x
Pass Criteria	R7 dedicated output functions correctly
Coverage Points	Special register functionality, indirect addressing support

3 ADDITIONAL TEST CASES

3.1 Boundary Value Arithmetic

Test ID	DP-07
Priority	P1
Title	Boundary Value Arithmetic
Objective	Test arithmetic operations at boundary conditions
Requirements	TR-ALU-05, TR-ISA-02, TR-ISA-03
Test Vectors	• 0xFF + 0x01 (unsigned overflow) • 0x7F + 0x01 (signed overflow) • 0x00 - 0x01 (unsigned underflow) • 0x80 - 0x01 (signed underflow) • NEG 0x80 (two's complement boundary)
Expected Results	Correct flag assertion for boundary conditions
Coverage Points	Overflow logic, flag boundary conditions

3.2 Register Write-Back Source Selection

Test ID	DP-08
Priority	P2
Title	Register Write-Back Source Selection
Objective	Verify rf_src_sel multiplexer functionality
Requirements	TR-REG-04
Test Steps	Test each rf_src_sel value: 00: ALU result (normal arithmetic) 01: RAM data (memory load) 10: Immediate value 11: I/O data (simulated) 100: SREG data (POP SREG)
Expected Results	Correct source selected for each rf_src_sel value
Coverage Points	Write-back multiplexer, data source routing

4 TEST EXECUTION PROCEDURE

4.1 Setup Instructions

1. Load Data Path testbench (tb_datapath.vhd)
2. Configure virtual sequencer as Control Unit surrogate
3. Initialize all registers to known state (default 0x00)
4. Set SREG to known value (default 0x00)
5. Configure clock generator: 10MHz, 50% duty cycle
6. Apply and release reset (active high, 2 clock cycles)

4.2 Execution Order

Order	Test Case	Dependencies	
1	DP-01	None (foundational)	
2	DP-02	DP-01 (requires working register file)	
3	DP-04	DP-02 (requires working ALU)	
4	DP-03	DP-04 (requires flag generation)	
5	DP-05	DP-02 (requires ALU routing)	
6	DP-06	DP-01 (requires register file)	
7	DP-07	DP-02, DP-04 (requires arithmetic)	
8	DP-08	All previous (comprehensive)	

4.3 Pass/Fail Criteria per Test

- **Pass:** All expected results match within 1 clock cycle tolerance
- **Fail:** Any mismatch in expected results
- **Warning:** Timing violations >1ns but functional behavior correct
- **Blocked:** Test cannot execute due to previous test failure

5 COVERAGE METRICS

5.1 Functional Coverage

Coverage Point	Description	Target
ALU Operations	All 11 operations from Table 3.3	100%
Flag Assertions	Each flag (H,V,N,C,Z) for each operation	100%
Register Access	Read/write to all 8 registers	100%
Operand Selection	All mux selection combinations	100%
Write-Back Sources	All rf_src_sel options	100%
Boundary Conditions	All arithmetic boundary cases	100%

5.2 Code Coverage Targets

- **Statement Coverage:** 95%
- **Branch Coverage:** 90%
- **Condition Coverage:** 85%
- **Path Coverage:** 90% (primary target for this stage)

6 SIGNAL MONITORING

6.1 Critical Signals to Monitor

Signal	Width	Monitoring Purpose
alu_result	8-bit	Verify arithmetic/logic results
flags	5-bit	Flag generation verification
data_a/data_b	8-bit	Register read verification
data_r7x	8-bit	X-pointer output verification
rf_src_sel	3-bit	Write-back source selection
alu_mux_sel_a/b	1-bit	Operand selection verification
regfile_we	1-bit	Write enable timing
sreg_we	1-bit	Status register update timing

6.2 Timing Checks

- Register write setup time: 2ns before clock edge
- Register write hold time: 1ns after clock edge
- ALU propagation delay: 10ns (combinational)
- Flag stabilization: Within 1 clock cycle

APPENDIX A: TEST VECTORS

6.3 Arithmetic Test Vectors

Listing 1: Sample Test Vectors for Arithmetic Operations

```
-- ADD operations
TEST_ADD_1: R1=0x10, R2=0x20, Result=0x30, Flags=00000
TEST_ADD_2: R1=0xFF, R2=0x01, Result=0x00, Flags=01011 (C=1,Z=1)
TEST_ADD_3: R1=0x7F, R2=0x01, Result=0x80, Flags=01100 (V=1,N=1)

-- SUB operations
TEST_SUB_1: R1=0x30, R2=0x10, Result=0x20, Flags=00000
TEST_SUB_2: R1=0x00, R2=0x01, Result=0xFF, Flags=01110 (C=0,N=1)

-- Logical operations
TEST_AND: R1=0xAA, R2=0x0F, Result=0x0A, Flags=00000
TEST_OR:  R1=0xA0, R2=0x0F, Result=0xAF, Flags=00000
TEST_XOR: R1=0xFF, R2=0xFF, Result=0x00, Flags=00010 (Z=1)
```

6.4 Boundary Value Test Vectors

Listing 2: Boundary Value Test Vectors

```
-- Maximum unsigned values
MAX_UNSIGNED_ADD: 0xFF + 0x01 = 0x00 (C=1, Z=1)
MAX_SIGNED_ADD:   0x7F + 0x01 = 0x80 (V=1, N=1)

-- Minimum signed values
MIN_SIGNED_SUB:   0x80 - 0x01 = 0x7F (V=1, N=0)
ZERO_NEG:         0x00 -> NEG -> 0x00 (Z=1, C=0)
MAX_NEG:          0x80 -> NEG -> 0x80 (V=1, N=1)
```

APPENDIX B: REQUIREMENTS TRACEABILITY

Test Case	Req ID	Requirement Description
DP-01	TR-REG-01	Verify asynchronous read on data_a and data_b
DP-01	TR-REG-02	Verify synchronous write to register when we is high
DP-02	TR-ALU-01	Verify all arithmetic/logical operations produce correct results
DP-02	TR-ISA-01	Verify ADD and ADC correctly sum two 8-bit registers
DP-03	TR-SREG-04	Verify bits I and T retain values during ALU flag updates
DP-04	TR-ISA-01	Verify ADC correctly sums with carry
DP-05	TR-ISA-08	Verify Immediate Load results in exact value in destination
DP-06	TR-REG-03	Verify data_r7x acts as dedicated hardwired output for R7
DP-06	TR-ISA-12	Verify Indirect Addressing accesses address in R7
DP-07	TR-ALU-05	Verify 2's complement handling in NEG and SUB operations
DP-07	TR-ISA-02	Verify ADD sets Carry Flag when result exceeds 255
DP-07	TR-ISA-03	Verify ADD sets Overflow Flag for signed overflow
DP-08	TR-REG-04	Verify Write Mux (rf_src_sel) correctly selects sources